

Here's a complete organizer briefing document:

Phantom Consensus — Organizer Briefing

What is this problem about?

Participants must build a Strategic Consensus Engine — a system that acts like a political advisor. Given a messy dataset of politicians, proposals, relationships, and objections, it must figure out:

Which proposals should be passed

Which politicians support the agreement

Which politicians are secretly allied with each other

Think of it like a UN Security Council simulator — you have representatives from different factions, each with their own agendas, trust levels, and hidden betrayal risks. Your engine must navigate all of this and produce a stable agreement.

What makes it hard?

Layer 1 — Dirty Data

The input is intentionally messy. Participants must clean it before they can even start:

"influence": "85" — a string instead of a number

"influence": null — missing value

"influence": 150 — out of range (max is 100)

"REP_001" vs "rep_001" vs " rep_001" — same person, different formats

Same proposal appearing twice with conflicting data

Ghost references — an objection from a rep who doesn't exist

Layer 2 — Feature Engineering

Raw values can't be used directly. Participants must derive meaningful features:

$\text{relationship_score} = \text{trust} \times (1 - \text{betrayal_prob})$ — a rep with trust=90 but betrayal=0.9 is actually dangerous

$\text{objection_weight} = \text{sum}(\text{severity} \times \text{objector_influence})$ — a powerful rep's objection matters more

$\text{proposal_viability} = \text{priority} \times (1 - \text{controversy})$ — a high-priority proposal everyone hates is not viable

Layer 3 — Multi-file Input

Data comes in 4 separate files in different formats:

representatives.json — JSON

proposals.json — JSON

objections.json — JSON

relations.csv — CSV (different format, must be parsed separately)

Layer 4 — Strategic Logic

After cleaning and engineering, the real challenge begins. The system must handle traps like:

Trojan Horse — A rep with influence=98 looks great. But betrayal_prob=0.95. Include them and the agreement collapses.

Poison Pill — A proposal with priority=10 looks critical. But every rep objects with severity=10. Select it and you have no supporters.

False Friend — Rep A trusts Rep B (trust=95). But Rep B doesn't trust Rep A (trust=25, betrayal=0.85). They should NOT be allied.

Faction Infiltrator — A rep claims to be in the Progressives faction. But they betray every Progressive with probability 0.90. Same faction ≠ safe ally.

What does the input look like?

Four files:

representatives.json — Who the politicians are

id, name, faction, influence (0-100)

proposals.json — What bills are on the table

id, title, sponsor (rep id), priority (1-10)

objections.json — Who opposes what

rep_id, proposal_id, severity (1-10)

relations.csv — How politicians feel about each other

from, to, trust (0-100), rivalry (0-100), betrayal_prob (0.0-1.0)

What does the output look like?

One JSON file:

```
{
  "final_agreement": {
    "proposals": ["prop_002", "prop_001"],
    "supporting_reps": ["rep_003", "rep_001", "rep_005"]
  },
  "alliances": [
    ["rep_001", "rep_004"],
    ["rep_002", "rep_005"]
  ]
}
```

proposals — which bills were selected

supporting_reps — which politicians back the agreement

alliances — which politicians are secretly working together

How is it evaluated?

Public Tests (participants can see these)

13 tests that check format only:

Is the output valid JSON?

Do all IDs exist in the input?

Are there duplicates?

Is there at least 1 proposal and 1 supporter?

These are easy to pass. Even a naive solution passes them.

Hidden Tests (only we run these)

18 scenario-based tests that check strategic correctness:

Scenario What it checks

Trojan Horse Did you exclude the high-betrayal rep?

Poison Pill Did you avoid the universally-objected proposal?

False Friend Did you detect asymmetric trust?

Clear Alliance Did you detect a genuine strong alliance?

Faction War Did you pick the least-objected proposal?

Priority vs Objection Did objection weight beat raw priority?

Supporter Coherence Did you avoid making objectors into supporters?

Faction Infiltrator Did you detect a spy in the faction?

Cascading Betrayal Did you exclude the high-risk end of a trust chain?

Alliance Hijack Did you protect a stable alliance from a disruptor?

Complete Rivalry Did you return empty alliances when everyone is a rival?

Ghost Sponsor Did you exclude proposals with non-existent sponsors?

Minimum Viable Does it work when only 1 valid rep and proposal remain?

ID Normalization Did you handle mixed-case IDs across files?

Duplicate Proposals Did you deduplicate correctly?

Null Influence Does it handle missing values without crashing?

Scale Correctness Does it make correct decisions on 50 reps, 30 proposals?

Dirty CSV Does it handle bad CSV rows without breaking clean ones?

Scoring

Each hidden test has 3 levels:

Full credit – correct decision, decisively

Partial credit – almost right but with mistakes

Zero – wrong decision

Total: 100 points

Tier Score Meaning

S 90-100 Exceptional – handles everything

A 75-89 Strong – handles most cases

B 60-74 Competent – basic strategy works

C 45-59 Acceptable – passes format, naive logic

D 30-44 Weak – barely functional

F 0-29 Failing

What separates a good solution from a bad one?

Bad solution (Tier C/D):

Sort proposals by priority → pick top 2

Sort reps by influence → pick top 3

Group reps with trust > 70 → call them alliances

This passes all public tests but fails most hidden tests. Score: ~35/100

Good solution (Tier A/B):

Clean all dirty data first

Compute $\text{relationship_score} = \text{trust} \times (1 - \text{betrayal_prob})$

Compute objection_weight per proposal

Exclude high-betrayal reps from supporters

Detect bidirectional alliances only

Handle edge cases gracefully

Score: ~75/100

Exceptional solution (Tier S):

Everything above, plus:

Detect faction infiltrators

Handle asymmetric trust correctly

Preserve stable alliances from disruptors

Make correct decisions at scale (50+ reps)

Normalize IDs across all 4 files

Score: 90+/100

What should participants submit?

consensus_engine.py

– their implementation

APPROACH.md – how they modeled the problem

RESULTS.md – analysis of their output

LIMITATIONS.md – what they couldn't handle

Dashboard or Visualization – visual representation of their solution

Common questions participants will ask

"What does stable mean?" We don't define it. That's intentional. They must decide.

"What betrayal_prob threshold should I use?" We don't say. That's part of the challenge.

"Can I add more files to src/?" Yes, as long as consensus_engine.py is the entry point.

"Can I use external libraries?" Yes, add them to requirements.txt. Keep it reasonable.

"Will the hidden tests use the same format?" Yes, same 4-file format. Different data, same structure.

Key things to know as an organizer

The hidden_tests/ folder is gitignored – participants never see it

The scoring engine runs on their actual submission (data/ + output/)

The correctness tests run on our crafted scenarios (self-contained)

A naive AI solution scores ~35/100 – the problem is designed to be AI-resistant

The dashboard is required but vague – used as tiebreaker for equal scores

No single correct answer exists – we reward strategic thinking, not formula-following